Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study an intended semantics-preserving discretization for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so inference on the generated artifact reduces to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Because this construction is global and type-directed, correctness requires reasoning beyond the local syntax . We formalize the operational semantics and state the measurable weak compiler simulation required for soundness of Boolean queries; completing that simulation remains an open proof obligation. Generated programs can be executed by discrete engines such as Dice, Roulette, and Storm. Our empirical evaluation studies case studies combining continuous sampling, higher-order functions, and unbounded recursion and benchmarks where direct comparison with existing systems is possible.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 118 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies an intended semantics-preserving source-to-source discretization that converts, if possible, continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\textbf{uniform}(0,1) < 0.5$ discretizes to $\textbf{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing exact inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

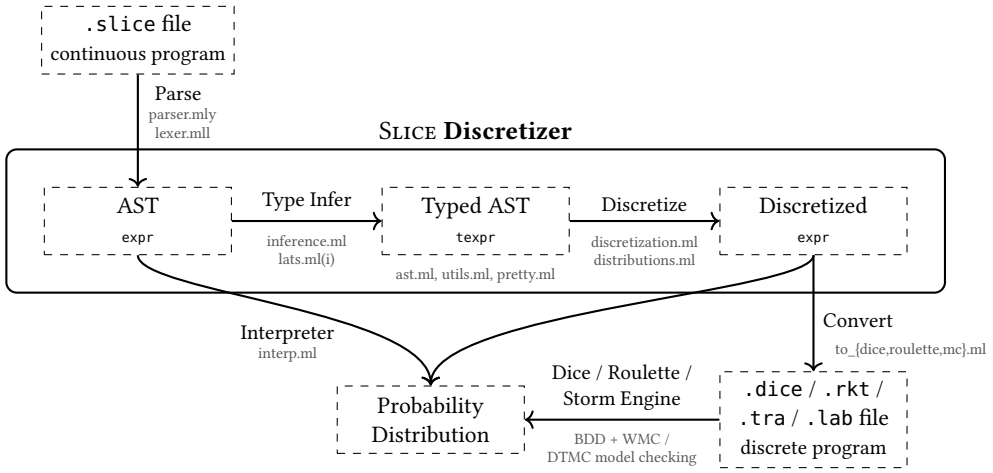


Fig. 1. Pipeline for discretization: typed Slice programs are transformed into discrete programs, then evaluated by discrete engines. Source-to-target equivalence is conditional on the weak compiler-simulation obligation in §6. The transformation stage (type inference + discretization) is backend-agnostic; Dice, Roulette, and Storm are examples of target backends.

propagates through each language construct (via one typing rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* names the intended equality of the relevant source and target probabilities. The abstract exact-real transfer theorem identifies sufficient semantic premises for equality. Whether the Slice transformation satisfies those premises remains conjectural. The OCaml/GSL binary64 artifact is not identified with the ideal exact-real target and is not certified by this theorem. This differs from approximate discretization, which intentionally changes the original semantics according to a precision parameter and may instead provide error bounds or convergence guarantees .

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.

Type Inference (§4) Infer types automatically to guide discretization.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

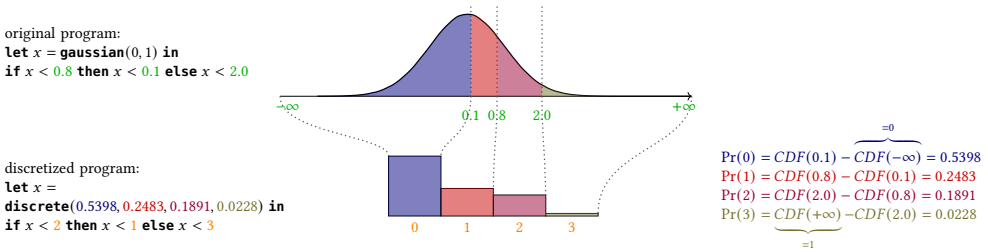


Fig. 2. Discretization of the original program into its semantically equivalent discretized program for example 2.1. The shaded regions show how the cuts at 0.1, 0.8, and 2.0 partition the continuous distribution into four intervals $(-\infty, 0.1)$, $[0.1, 0.8)$, $[0.8, 2.0)$, and $[2.0, +\infty)$, whose masses become the probabilities of outcomes 0, 1, 2, and 3, i.e., interval indices, in the discrete distribution.

Soundness (§6) Prove semantic preservation of the discretization transformation with a coupling-style relational argument [7, 13, 48].

Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized programs. We provide (i) case studies combining continuous sampling, higher-order functions, and unbounded recursion; to the best of our knowledge, there is no prior exact inference system that can cover programs *combining* all three of these features and (ii) case studies where Slice+Dice and Slice+Roulette show favorable scaling and empirical performance on new and existing benchmarks [22, 40].

2 Slice by Example

To build intuition for Slice, we start with simple examples demonstrating the key ideas, and then provide a more involved example containing higher-order recursion and lists in §2.6. Our pipeline automatically produces and analyzes discrete artifacts for all provided programs; source-to-target equality remains conditional on the weak compiler-simulation obligation in §6.

2.1 Branching on a Single Continuous Variable

Consider a program that samples a real value from a standard Gaussian distribution and then performs two nested comparisons: first checking whether the sample is less than 0.8, then comparing it to either 0.1 or 2.0:

```
let x = gaussian(0,1) in if x < 0.8 then x < 0.1 else x < 2.0
```

We discretize this program by replacing the real-valued sample with a discrete interval index, as shown below and in Figure 2:

```
let x = discrete(0.5398, 0.2483, 0.1891, 0.0228) in if x < 2 then x < 1 else x < 3
```

Here, **discrete**(0.5398, 0.2483, 0.1891, 0.0228) already represents all four observationally distinguishable outcomes of the Gaussian sample. The precise real value of x never matters to the rest of the program; only the interval in which it falls does. This is the regime we target: continuous quantities observed only through finitely many comparison outcomes. *Slice's goal is to detect such situations automatically.* We partition the continuous distribution into intervals that are indistinguishable to the rest of the program, then map every real to the interval it inhabits.

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a gaussian, then use
   the resulting value to update flags *)
let step = fun ys ->
  update (gaussian(0,1)) ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 4. An infinite stochastic process for random interval covering over $[0, 1]$.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a discrete program (shown in §A.5); its equivalence to the source is conditional on the weak compiler-simulation obligation in §6. Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Probabilistic model checking then computes the result for the generated artifact: we use Storm [16], which reports 0.034474.... See §7.1 for details. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\mid \textbf{discrete}(p_0, \ldots, p_n) & \text{discrete distribution} \\
\mid e_1 < e_2 \mid e_1 \leq e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \leq_{\#n} e_2 \mid e_1 ==_{\#n} e_2 & \text{comparisons} \\
\mid e_1 \,\&\&\, e_2 \mid e_1 \,||\, e_2 \mid \text{not } e & \text{Boolean operations} \\
\mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\mid \textbf{fun}\ x\ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with}\ \mid \text{nil} \to e_{\text{nil}}\ \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct $\textbf{observe}(e)$ conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{{<}x \mid x \in \mathbb{R}\} \cup \{{\leq}x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a non-empty cut set $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ with $x_1 \leq \ldots \leq x_n$, we define the set $\text{Intervals}(B)$ of intervals it induces as expected, e.g., $\text{Intervals}(\{{<}0.1, {\leq}0.5\}) = \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: $\text{Intervals}(\{{<}c\}) = \{(-\infty, c), [c, \infty)\}$, whereas $\text{Intervals}(\{{\leq}c\}) = \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \leq c$ places c on the left. Moreover, we let $\text{Intervals}(\emptyset) = \{(-\infty, \infty)\}$ and $\text{Intervals}(\top) = 2^{\mathbb{R}}$. We denote by $\text{Intervals}(B)_i$ the i-th interval of $\text{Intervals}(B)$, assuming $\text{Intervals}(B)$ is 0-indexed, i.e., $i \in \{0, ..., n\}$. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Definition 3.1 (Types). For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types τ adhere to the grammar

$$\tau ::= \textbf{int} \mid \textbf{bool} \mid \textbf{unit} \mid \textbf{fin}(n) \mid \textbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \textbf{list}(\tau)\ .$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\textbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\textbf{float}[B; V]$ annotates a real-valued expression with its cut set B (an over-approximation of the comparisons it participates in) and value set V (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the $\text{answers}_<(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_<(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {<}x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {\leq}x \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates*. That is, if B contains ${<}x$ then the discretized value of $e : \texttt{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \texttt{float}[B; V_1]$ and $e_2 : \texttt{float}[B; V_2]$, then $\text{answers}_<(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all ${<}x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_\leq(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {\leq}x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {<}x \in B$.

Subtyping. The type system includes a subtyping relation that captures when one type can be replaced by another. The subtyping rules are shown in Figure 8. For float types $\texttt{float}[B; V]$, subtyping follows the lattice ordering on V: an expression known to range over some set of values can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\texttt{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 : \tau}$$

$$\textsc{Observe}\ \frac{\Gamma \vdash e : \textbf{bool}}{\Gamma \vdash \textbf{observe}\ e : \textbf{unit}} \qquad \textsc{Unit}\ \frac{}{\Gamma \vdash () : \textbf{unit}} \qquad \textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\, e_2 : \tau_2}$$

$$\textsc{Fix}\ \frac{\Gamma, f : \tau_1 \to \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \textbf{fix}\ f\ x := e : \tau_1 \to \tau_2} \qquad \textsc{Discrete}\ \frac{}{\Gamma \vdash \textbf{discrete}(p_0, \ldots, p_{n-1}) : \textbf{fin}(n)} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \textbf{fin}(n)}$$

$$\textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \textbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \textbf{bool}}$$

$$\textsc{FinEq}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 ==_{\#n} e_2 : \textbf{bool}} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \textbf{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

$$\textsc{Sub-Refl}\ \frac{\tau \in \{\textbf{int}, \textbf{bool}, \textbf{fin}(n)\}}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\textbf{float}[B_1; V_1] <: \textbf{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'}$$

$$\textsc{Sub-List}\ \frac{\tau <: \tau'}{\textbf{list}(\tau) <: \textbf{list}(\tau')} \qquad \textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

The distinction between typing and type inference is important. The intended correctness argument separates into two obligations: (1) type inference produces a typing derivation matching our typing rules, and (2) given such a typing derivation, the generated program satisfies the weak compiler simulation stated in §6. The second obligation is not discharged here.

4.1 Constraint Generation

We use the constraint language in Figure 9 to express requirements over the type structure and float refinements. We adopt the framework of [38], introducing a new judgment of the form $\Gamma \vdash e : \tau \mid C$, which reads as *in the environment* Γ, *e has type* τ *and produces the set of constraints* C. In this judgment, Γ and e are inputs to the type inference algorithm, while τ and C are outputs: the algorithm infers the type τ for expression e and generates the constraints C that must hold for this typing to be valid. The inference rules formalize which constraints must be generated and later solved for a given expression e to have inferred type τ. Figure 10 gives the constraint generation rules for Slice. We focus on a small but representative subset of constructs in Slice, capturing the key ideas behind constraint generation for the *full* language.

The type inference rules closely mirror the typing rules in Figure 6, with two key differences. First, the inference rules generate a constraint set instead of having those constraints appear in the premises. Second, subtyping is handled in a syntax-directed way: rather than using a general subtyping rule, we bake the subtyping relation into each rule.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\mathbf{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\mathbf{float}[B; V] \to \mathbf{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression $e : \mathbf{bool}$ and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}$ such that e and $\mathcal{D}[\![e : \mathbf{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this intended semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. The abstract exact-real transfer theorem identifies sufficient semantic premises for equality. Whether the Slice transformation satisfies those premises remains conjectural. The OCaml/GSL binary64 artifact is not identified with the ideal exact-real target and is not certified by this theorem. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization aims to make that reduction explicit.

The displayed rules define the ideal exact-real target transformation. For a finite dispatcher, an index not represented by the enumerated arms, or an enumerated parameter tuple outside the formal domain of the sampler, takes the primitive target term **diverge**. This is distinct from **observe** false: the former contributes missing mass, whereas the latter produces explicit observation-failure mass at $\bot$.

Correct branch selection additionally requires the reachable-value adequacy premise that every runtime parameter value belongs to its inferred value set. The final finite-equality guard rejects unmatched indices, but does not prove this adequacy premise.

The OCaml emitter uses hygienic left-to-right bindings, nested parameter-by-parameter dispatch, and represents primitive divergence by $(\mathbf{fix}\ f\ u := f\ u)\ ()$. This PR does not prove that the literal emitted AST implements the ideal rule, nor that the recursive encoding is weakly cofinal with primitive divergence.

Lemma 5.1 (Typing of generated recursive divergence). *For fresh f, u and every type τ,*

$$\Gamma \vdash (\mathbf{fix}\ f\ u := f\ u)\ () : \tau.$$

Proof. Under $f : \mathbf{unit} \to \tau$ and $u : \mathbf{unit}$, application gives $f\ u : \tau$. Hence Fix gives $\mathbf{fix}\ f\ u := f\ u : \mathbf{unit} \to \tau$. The result follows from Unit and App. □

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only subexpressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; the

$e : \tau$	Condition	$\mathcal{D}[\![e : \tau]\!]$
$x : \tau$		x
$c : \texttt{float}[B; V]$	$B = \top$	c
	$B \neq \top$	$k_{\#(\lvert B\rvert+1)}$ such that $c \in \text{Intervals}(B)_k$
$\texttt{let}\ x = e_1 : \tau_1\ \texttt{in}\ e_2 : \tau_2 : \tau_2$		$\texttt{let}\ x = \mathcal{D}[\![e_1 : \tau_1]\!]\ \texttt{in}\ \mathcal{D}[\![e_2 : \tau_2]\!]$
$(\texttt{if}\ e_1 : \texttt{bool}\ \texttt{then}\ e_2 : \tau\ \texttt{else}\ e_3 : \tau) : \tau$		$\texttt{if}\ \mathcal{D}[\![e_1 : \texttt{bool}]\!]\ \texttt{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \texttt{else}\ \mathcal{D}[\![e_3 : \tau]\!]$
$(e_1 : \texttt{float}[B; V_1] < e_2 : \texttt{float}[B; V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] <_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] < \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
$(e_1 : \texttt{float}[B; V_1] \leq e_2 : \texttt{float}[B; V_2]) : \texttt{bool}$	$B \neq \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] \leq_{\#(\lvert B\rvert+1)} \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
	$B = \top$	$\mathcal{D}[\![e_1 : \texttt{float}[B; V_1]]\!] \leq \mathcal{D}[\![e_2 : \texttt{float}[B; V_2]]\!]$
$\texttt{uniform}(e_1 : \texttt{float}[B_1; V_1],\ e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B; V]$	$e_1, e_2 \in \mathbb{R},\ e_1 \geq e_2$	$\texttt{diverge} : \mathcal{D}[\![\texttt{float}[B; V]]\!]$
	$B = \top,\ \neg(e_1, e_2 \in \mathbb{R} \wedge e_1 \geq e_2)$	$\texttt{uniform}(\mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!])$
	$e_1, e_2 \in \mathbb{R},\ e_1 < e_2,$ $V_1 = \{e_1\},\ V_2 = \{e_2\},$ $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$	$\texttt{discrete}(p_0, \ldots, p_n)$ where $p_k = \Pr[X \in \text{Intervals}(B)_k],\ X \sim \text{Uniform}(e_1, e_2)$
	$B \neq \top,\quad (e_1 \notin \mathbb{R}$ or $e_2 \notin \mathbb{R}),$ $B_1 \neq \top,$ $B_2 \neq \top,$ $V_1 = \{v_1, \ldots, v_n\},$ $V_2 = \{w_1, \ldots, w_m\}$	$\texttt{let}\ v_1' = \mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!]\ \texttt{in}$ $\texttt{let}\ v_2' = \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!]\ \texttt{in}$ $\texttt{if}\ v_1' =_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_1 : \texttt{float}[B_1; V_1]]\!]$ $\quad \&\&\ v_2' =_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_1 : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_1, w_1) : \texttt{float}[B; V]]\!]$ $\ldots$ $\texttt{else if}\ v_1' =_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_i : \texttt{float}[B_1; V_1]]\!]$ $\quad \&\&\ v_2' =_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_j : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_i, w_j) : \texttt{float}[B; V]]\!]$ $\ldots$ $\texttt{else if}\ v_1' =_{\#(\lvert B_1\rvert+1)} \mathcal{D}[\![v_n : \texttt{float}[B_1; V_1]]\!]$ $\quad \&\&\ v_2' =_{\#(\lvert B_2\rvert+1)} \mathcal{D}[\![w_m : \texttt{float}[B_2; V_2]]\!]\ \texttt{then}$ $\quad\quad \mathcal{D}[\![\texttt{uniform}(v_n, w_m) : \texttt{float}[B; V]]\!]$ $\texttt{else diverge} : \texttt{fin}(\lvert B\rvert + 1)$

Fig. 13. Discretization function. The displayed cases are pairwise disjoint. Uniform distributions require a strict parameter ordering $e_1 < e_2$. Equivalently, because such distributions are atomless, the interval masses in the constant-parameter rule can be computed from the clamped CDF, with $\text{CDF}(-\infty) = 0$ and $\text{CDF}(+\infty) = 1$. The final row is the ideal exact-real dispatch schema. The OCaml emitter uses a nested hygienic core elaboration; correspondence between that emitted term and this schema is an explicit compiler-correctness obligation. For the **uniform** distribution, $V = \top$ is implicit, and all remaining cut-set and value-set cases are outside this rule. The theorem applies only under the displayed representation, finite-domain, nonempty-domain, sampler-domain, and reachable-value adequacy premises. Equal or reversed uniform bounds are malformed under the strict uniform contract. For all other language constructs, $\mathcal{D}[\![\cdot]\!]$ recurses structurally over them.

appendix lists the outstanding compiler-simulation conjectures for this non-local, type-mediated construction . The implementation represents finite equality with the same $==_{\#n}$ core expression used by the ideal schema: its type checker requires two $\texttt{fin}(n)$ operands and its evaluator compares their finite indices. This shared primitive does not by itself prove correspondence between the flattened schema and the emitter's nested dispatch. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \texttt{uniform}(e_1, e_2)$ of type $\texttt{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression $e' : \texttt{fin}(n+1)$ such that the probability that e evaluates to a value in $\text{Intervals}(B)_i$

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{}{\Gamma \vdash \textbf{discrete}(p_0, \ldots, p_{n-1}) : \textbf{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e : \textbf{int}}{\Gamma \vdash e \le i : \textbf{bool}} \qquad \textsc{Pair}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 * \tau_2}$$

$$\textsc{Fst}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \textbf{fst}\ e : \tau_1} \qquad \textsc{Snd}\ \frac{\Gamma \vdash e : \tau_1 * \tau_2}{\Gamma \vdash \textbf{snd}\ e : \tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \textbf{fun}\ x\ \rightarrow\ e : \tau_1 \rightarrow \tau_2} \qquad \textsc{App}\ \frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

$$\textsc{True}\ \frac{}{\Gamma \vdash \text{true} : \textbf{bool}} \qquad \textsc{False}\ \frac{}{\Gamma \vdash \text{false} : \textbf{bool}} \qquad \textsc{And}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \textbf{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \textbf{bool}}$$

$$\textsc{Or}\ \frac{\Gamma \vdash e_1 : \textbf{bool} \qquad \Gamma \vdash e_2 : \textbf{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \textbf{bool}} \qquad \textsc{Not}\ \frac{\Gamma \vdash e : \textbf{bool}}{\Gamma \vdash \text{not}\ e : \textbf{bool}} \qquad \textsc{Unit}\ \frac{}{\Gamma \vdash () : \textbf{unit}} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \textbf{fin}(n)}$$

$$\textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \textbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \textbf{bool}}$$

$$\textsc{FinEq}\ \frac{\Gamma \vdash e_1 : \textbf{fin}(n) \qquad \Gamma \vdash e_2 : \textbf{fin}(n)}{\Gamma \vdash e_1 ==_{\#n} e_2 : \textbf{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \textbf{bool}}{\Gamma \vdash \textbf{observe}\ e : \textbf{unit}} \qquad \textsc{Seq}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1; e_2 : \tau_2}$$

$$\textsc{Fix}\ \frac{\Gamma, f : \tau_1 \rightarrow \tau_2, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fix}\ f\ x := e : \tau_1 \rightarrow \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \text{nil} : \textbf{list}(\tau)} \qquad \textsc{Cons}\ \frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \textbf{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \textbf{list}(\tau)}$$

$$\textsc{Match}\ \frac{\Gamma \vdash e : \textbf{list}(\tau_1) \qquad \Gamma \vdash e_1 : \tau_2 \qquad \Gamma, h : \tau_1, t : \textbf{list}(\tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \text{match}\ e\ \text{with nil} \rightarrow e_1 \mid h :: t \rightarrow e_2\ \text{end} : \tau_2}$$

$$\textsc{Diverge}\ \frac{}{\Gamma \vdash \textbf{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

Case $e = (e_1 : \textbf{float}[B; V_1] \le e_2 : \textbf{float}[B; V_2])$. Analogous to the $<$ case, using $\text{answers}_{\le}(B, V_1, V_2)$ and FinLessEq.

Case $e = (\textbf{uniform}(e_1 : \textbf{float}[B_1; V_1],\ e_2 : \textbf{float}[B_2; V_2]) : \textbf{float}[B; \top])$. The Continuous-2 rule was applied, so $\text{recovers}(B_1, V_1)$ and $\text{recovers}(B_2, V_2)$ hold. By the inductive hypothesis,

- Consider $e = \textbf{diverge} : \tau$.
  The small-step semantics is:

$$[\![\textbf{diverge} : \tau]\!] = \lambda A.\, 0.$$

  By Lemma F.32,

$$\lambda A.\, 0$$

  is a Markov kernel.
  Thus, $[\![\textbf{diverge} : \tau]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ is a Markov kernel.
- All other cases are analogous.

□

The former synchronous proof is omitted. It relied on a false translation/substitution commutation equation and could not account for administrative stuttering or for the recursive implementation encoding of divergence.

F.5 Outstanding Slice compiler-simulation conjectures

The legacy main-text summary still cites these labels. They now point to this list of open obligations, not to established lemmas or synchronous coupling tables.

The abstract exact-real transfer theorem applies to Slice only after the following premises have been established for the actual source and target kernels:

(1) a measurable related-state carrier R;
(2) one coherent measurable joint sub-Markov kernel $C : R \to \mathcal{D}_{\leq 1}(R)$;
(3) exact source and weak-target marginals for C;
(4) terminal synchrony, with Boolean values related diagonally and observation failure related only to observation failure; and
(5) weak-to-ordinary target terminal cofinality.

Pointwise existence of couplings is insufficient. In addition, the emitted recursive encoding of **diverge** has a deterministic self-step and therefore cannot be coupled one-for-one with the primitive zero kernel. Its treatment requires a divergence-delay clause and a separate terminal-cofinality proof. All Slice-specific premises in this subsection remain conjectures.

G Benchmarks

G.1 Asymptotic Scaling

G.1.1 Code Sketches for Scaling Benchmarks. This section contains code snippets of the programs used in the scaling experiments in Section 7.2. We implemented a Python script that automatically generates these programs based on configurable parameters such as program size, guard structure, and dependency pattern. For reproducibility, we used a fixed random seed to generate each program. All r_i values in the generated programs, where $0 \leq r_i \leq 1$ is randomly chosen, are rounded to two decimal places.

All of these programs follow the same structural pattern: they define a sequence of random variables $X_1, \ldots, X_n$, where each X_i is drawn from a uniform distribution whose parameters depend on the value of a single predecessor variable through a conditional branch of the form

$$X_i = \begin{cases} \mathrm{Unif}(a_i, b_i), & X_{\pi(i)} < r_i, \\ \mathrm{Unif}(a_i', b_i'), & \text{otherwise}, \end{cases}$$